

QA Executive Summary

Demoblaze E-Commerce Platform — UI Automation Test Execution Report

EXECUTION DATE May 2, 2026	ENVIRONMENT Staging	BROWSER Chromium	FRAMEWORK Playwright + TypeScript
-------------------------------	------------------------	---------------------	--------------------------------------

3 TESTS PASSED Functionally verified	0 UNEXPECTED FAILURES Suite is stable	1 KNOWN BUG TRACKED Intentional · test.fail()	~19s WALL-CLOCK TIME 4 workers in parallel
--	---	---	--

≡ Test Results

TEST SCENARIO	SUITE FILE	DURATION	WORKER	STATUS	NOTES
Product Catalog Scraping	scenarioA.spec.ts	~7s	worker-2	PASSED	3 toPass retries on pagination
E2E Purchase Flow	scenarioB.spec.ts	~13s	worker-3	PASSED	Fixture-based auth + cart cleanup
Full User Journey	fullUserJourney.spec.ts	~17s	worker-1	PASSED	Signup → Login → 2 products → delete → purchase
Checkout Negative Test	checkoutNegative.spec.ts	~19s	worker-0	KNOWN BUG	Intentional fail — no CC validation in app

! Failed Tests & Root Cause Analysis

BUG-001 · Checkout form accepts invalid credit card numbers

📄 checkoutNegative.spec.ts 🔧 CartPage.assertCreditCardValidationError() ⌚ Timeout: 10,000ms ⚠️ Severity: Normal

What the test does: Submits the checkout form with an invalid credit card number ("1234") and asserts that a validation error message appears inside the Order modal.

What the application does: Demoblaze performs no frontend validation on the credit card field. Upon clicking Purchase, the modal closes and the purchase success dialog (.sweet-alert) is displayed immediately, regardless of the input value.

Technical root cause: The assertion `expect(locator('#orderModal').getByText('Invalid credit card number')).toBeVisible()` waited 10,000ms for an element that is never rendered. The Order modal was already dismissed by the time the assertion ran, leaving the locator with zero matching elements in the DOM. The Playwright error log confirms: *"element(s) not found — waiting for locator('#orderModal').getByText('Invalid credit card number')"*.

Framework handling: The test is decorated with `test.fail(true, '...')`, which inverts the pass/fail signal. The test is therefore reported as **passed** in CI and will automatically flip to red once the application implements the expected validation, serving as a regression guard.

Pagination Retries in Scenario A (non-blocking)

The `goToNextPage()` method detected 3 internal retries via `expect().toPass()` before the page-2 product list rendered. Demoblaze's pagination updates the DOM asynchronously (~900ms after the Next click), temporarily leaving the same product names from page 1 visible. The polling strategy handled this correctly and the test passed without retry. This is informational — no action required.

🔗 Detected Patterns

Stable Authentication Flows

Sign-up and login operations completed consistently across all three tests that require authentication (Scenario B fixture, Full User Journey). The average auth sequence (sign-up dialog → login modal → nav bar confirmation) completed in **2–3 seconds** with no flakiness observed. Unique usernames generated via `Date.now()` prevented any account collision under parallel execution.

Cart Operations Are Reliable

Add-to-cart (network-synced via `waitForResponse('**/addtocart')`), item deletion (DOM-synced via `not.toBeAttached()`), and total recalculation (polled via `toHaveText(/\\d+/)`) all performed reliably across parallel runs. No race conditions were observed in cart state management.

Page Load Latency (~4–5s) is an External Dependency

All four tests navigate to the Demoblaze homepage and consistently wait ~4–5 seconds for the product grid to render. This latency originates from the application's CDN/server response time and is outside the control of the automation framework. It is stable and predictable — not a flakiness concern — but it sets a floor on how fast any test can start its business logic.

Pagination Rendering Is Slow but Consistently Handled

Demoblaze's catalog pagination has no loading indicator and updates the DOM approximately 900ms after the Next button click. The `toPass()` polling pattern catches this reliably, but it represents a fragile contract with the application's frontend performance. A regression in page-2 load time could increase retry count or cause timeout.

Parallel Execution Shows No Data Interference

All 4 tests launched simultaneously (same epoch millisecond in the Allure trace) and completed without interfering with each other. The data isolation strategy — unique users per test, independent cart state, `afterEach` cleanup in Scenario B — proved effective. Wall-clock time of ~19s vs. ~47s sequential confirms a **~64% reduction** in execution time.

✓ Action Suggestions

DEVELOPMENT TEAM

- HIGH** **Implement credit card validation on the checkout form (BUG-001).** The purchase form should reject card numbers that fail the Luhn algorithm or fall outside accepted length ranges before submitting the order. Validation should occur client-side on form submission and display an error message within the Order modal without closing it. Once implemented, the `checkoutNegative` test will automatically become a passing regression guard.
- MED** **Add a loading indicator to catalog pagination.** The current Next/Prev navigation updates the DOM asynchronously with no visual feedback, which makes both manual UX and automation synchronization harder than necessary. A simple CSS class toggle or spinner during the load would allow the framework to use a deterministic wait instead of polling.
- LOW** **Expose a consistent network event for cart updates.** The `DELETE` cart item operation does not reliably emit a `viewcart` response, which forced the framework to fall back to DOM-level polling. A predictable API contract (e.g., consistent response after deletion) would allow cleaner, faster synchronization.

QA TEAM

- HIGH** **Expand negative test coverage for the checkout form.** Once BUG-001 is fixed, add test cases for: empty required fields, expired card dates, missing card number, and special characters in text fields. The `invalidCreditCardDetails` fixture in `testData.ts` can be extended with additional datasets using Playwright's `test.each()`.
- MED** **Add cross-browser coverage (Firefox, WebKit).** The current suite runs exclusively on Chromium. Adding Firefox and WebKit projects to `playwright.config.ts` would surface any browser-specific rendering or behavior differences, especially around dialog handling and modal animations.
- MED** **Add a pagination stability guard to Scenario A.** The `toPass()` polling currently has no configurable timeout separate from the global test timeout. Consider wrapping the pagination assertion with an explicit `{ timeout: 5000 }` option so slow pagination fails fast and is clearly attributed to the correct cause in the report.
- LOW** **Configure Allure history to track flakiness trends over time.** Allure supports historical trend charts when `allure-results/history` is preserved between runs. In the GitHub Actions workflow, cache or artifact the history folder so retries and pass-rate trends become visible across pipeline executions.